

feature of Cryptomator. With it, you don't have to deal with accounts, key management, cloud access grants or cipher configurations. Just with a password, one can use the vault and store data on any cloud. Users are not required to specify which cloud network to use. Cryptomator uses transparent data encryption and makes it very easy to store data on any cloud. It encrypts files and doesn't care where you store them. This makes it a lightweight application that is easy to run on any computer or smartphone, reliably. Cryptomator also has a feature called Sanitizer, with which it is possible to scan for any corrupted files in the vault. This feature helps troubleshoot any error and makes Cryptomator more reliable in terms of data protection.

Functionality

Cryptomator creates a virtual drive (vault) to add, edit, and remove files, as is the case with just about any disk drive. Files are encrypted using a method called Transparent Data Encryption (TDE). The encrypted data is converted to unreadable code before storing it on the cloud, and no unencrypted copies are stored on the hard drive. For every encryption, a master key is created and stored separately. The encrypted files are decrypted and converted back to readable format using the master key. Every time your files are accessed inside the virtual drive, Cryptomator will encrypt/decrypt them on-the-fly.

Currently, Dokany (on Windows) and FUSE (on MacOS and Linux) are the frontends of choice. If they are not available on your system, Cryptomator will fall back on WebDAV, as it is supported on every major operating system. WebDAV is an HTTP based protocol, and Cryptomator will act as a WebDAV server accepting so-called loopback connections on your computer.

Whenever your file manager accesses files through this virtual

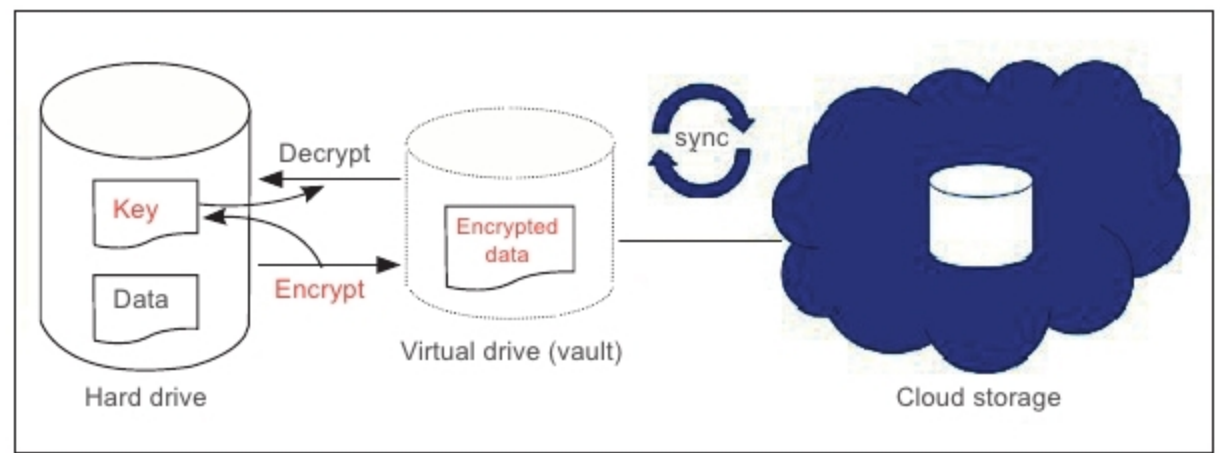


Figure 1: Transparent data encryption

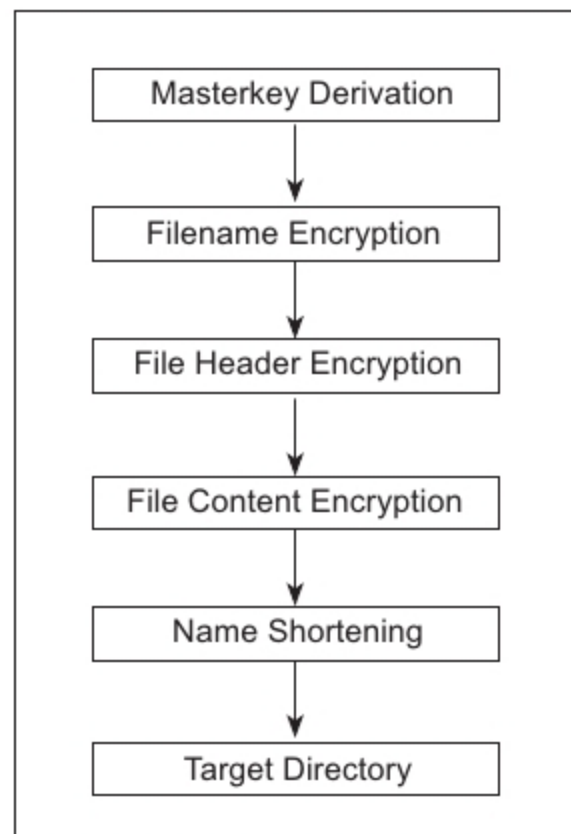


Figure 2: Cryptomator's six layers of encryption and storage methods

drive, Cryptomator will process this request via the following six layers of encryption and storage methods.

Master key derivation: This is the first layer of encryption and here, the master key is created for the file that is being stored in the vault. Every vault has its own 256-bit encryption key, and both the keys are encrypted using RFC 3394 key wrapping with a KEK derived from the user's password, using Scrypt. The encrypted keys are stored as integers or Base64 strings in a JSON file named *masterkey.cryptomator*, which is located in the root directory of the vault. When unlocking a vault, the KEK is used to unwrap (i.e., decrypt) the stored master keys.

File name encryption: In this layer, the file and directory names are first encrypted before storing. Initially, each folder gets a unique identifier called the directory ID. The directory ID for the root folder is special and always empty. For all other folders a UUID is created. Cryptomator uses AES-SIV to encrypt files as well as directory names and associates according to the ID.

File header encryption: In this layer, the file header stores certain metadata, which is needed for file content encryption. It consists of 88 bytes:

- 16 byte nonce used during header payload encryption
- 40 byte AES-CTR encrypted payload that includes 8 bytes filled with one for future use (formerly used for the file size) and 32 byte file content key
- 32 byte header MAC of the previous 56 bytes

File content encryption: This is where the actual file contents get encrypted. The cleartext is broken down into multiple chunks, each up to 32KB + 48 bytes consisting of:

- Up to 32KB encrypted payload using AES-CTR with the file content key
- 16 byte nonce
- 32 byte MAC of file header nonce (to bind this chunk to the file header)
- Chunk number as 8 byte big endian integer (to prevent undetected reordering)
- Nonce
- Encrypted payload